

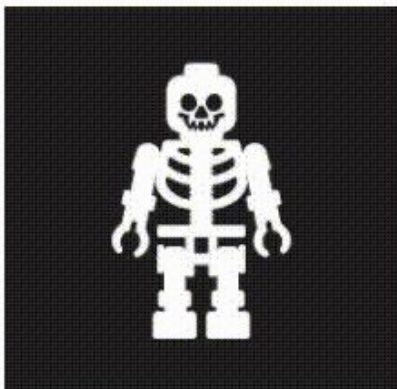


JavaScript



What is JavaScript

HTML
structure



CSS
presentation/appearance



JavaScript
dynamism/action



History

JavaScript was invented by Brendan Eich in 1995. It was developed for Netscape 2, and became the ECMA-262 standard in 1997. After Netscape handed JavaScript over to ECMA, the Mozilla foundation continued to develop JavaScript for the Firefox browser.



What is JavaScript

JavaScript is the world's most popular programming language.

JavaScript is the programming language of the Web.

JavaScript is easy to learn.

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>My First JavaScript</h2>
```

```
<button type="button"
onclick="document.getElementById('demo').innerHTML = Date()">
Click me to display Date and Time.</button>
```

```
<p id="demo"></p>
```

```
</body>
</html>
```

My First JavaScript

Click me to display Date and Time.

My First JavaScript

Click me to display Date and Time.

Wed Sep 08 2021 11:21:29 GMT+0530 (India Standard Time)

JavaScript Can Change HTML Content

JavaScript accepts both double and single quotes:

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change HTML content.</p>
```

```
<button type="button" onclick='document.getElementById("demo").innerHTML = "Hello  
JavaScript!'">Click Me!</button>
```

```
</body>  
</html>
```

What Can JavaScript Do?

JavaScript can change HTML content.

Click Me!

What Can JavaScript Do?

Hello JavaScript!

Click Me!

JavaScript Can Change HTML Attribute Values

What Can JavaScript Do?

JavaScript can change HTML attribute values.

In this case JavaScript changes the value of the src (source) attribute of an image.



Turn on the light

Turn off the light

What Can JavaScript Do?

JavaScript can change HTML attribute values.

In this case JavaScript changes the value of the src (source) attribute of an image.



Turn on the light

Turn off the light

JavaScript Can Change HTML Attribute Values

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<!-- <button onclick="document.getElementById('myImage').src='start.gif'">Turn on the light</button> -->  
<input type="button" value="Click me" onclick="document.getElementById('myImage').src='start.gif'">
```

```

```

```
<!-- <button onclick="document.getElementById('myImage').src='stop.png'">Turn off the light</button> -->  
<input type="button" value="Click me" onclick="document.getElementById('myImage').src='stop.png'">
```

```
</body>  
</html>
```

Single Button using on & off using function

```
<!DOCTYPE html>
<html>
<body>

<button onclick="toggleLight()">Toggle the light</button>



<script>
var isLightOn = false; // Initial state is off

function toggleLight() {
  var imageElement = document.getElementById('myImage');
  if (isLightOn) {
    // If the light is currently on, turn it off
    imageElement.src = 'pic_bulboff.gif';
  } else {
    // If the light is currently off, turn it on
    imageElement.src = 'pic_bulbon.gif';
  }
  // Toggle the state
  isLightOn = !isLightOn;
}
</script>

</body>
</html>
```

```
<script>

function toggleLight() {
  var imageElement =
document.getElementById('myImage');

  if (imageElement.src.endsWith('stop.png')) {
    // If the image source ends with
    'stop.png', change it to 'start.gif'
    imageElement.src = 'start.gif';
  } else {
    // If the image source doesn't end with
    'stop.png', change it to 'stop.png'
    imageElement.src = 'stop.png';
  }
}

</script>
```

JavaScript Can Change HTML Styles (CSS)

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can change the style of an HTML element.</p>
```

```
<button type="button" onclick="document.getElementById('demo').style.fontSize='35px'">Click Me!</button>
```

```
</body>
```

```
</html>
```

What Can JavaScript Do?

JavaScript can change the style of an HTML element.

Click Me!

What Can JavaScript Do?

JavaScript can change the style of an HTML element.

Click Me!

JavaScript Can add multiple Styles (CSS)

```
<!DOCTYPE html>
<html>
<body>

<h2>What Can JavaScript Do?</h2>

<p id="demo">JavaScript can change the style of an HTML element.</p>

<button type="button" onclick="styleText()">Click Me!</button>

<script>
function styleText() {
    var element = document.getElementById('demo');
    element.style.fontSize = '35px';
    element.style.color = 'blue'; // Change text color
    element.style.backgroundColor = 'yellow'; // Change background color
}
</script>

</body>
</html>
```

JavaScript Can Hide HTML Elements

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p id="demo">JavaScript can hide HTML elements.</p>
```

```
<button type="button" onclick="document.getElementById('demo').style.display='none'">Click Me!</button>
```

```
</body>
```

```
</html>
```

What Can JavaScript Do?

JavaScript can hide HTML elements.

Click Me!

What Can JavaScript Do?

Click Me!

JavaScript Can Show HTML Elements

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>What Can JavaScript Do?</h2>
```

```
<p>JavaScript can show hidden HTML elements.</p>
```

```
<p id="demo" style="display:none">Hello JavaScript!</p>
```

```
<button type="button" onclick="document.getElementById('demo').style.display='block'">Click Me!</button>
```

```
</body>
```

```
</html>
```

What Can JavaScript Do?

JavaScript can show hidden HTML elements.

Click Me!

What Can JavaScript Do?

JavaScript can show hidden HTML elements.

Hello JavaScript!

Click Me!

Single Click to Hide & Show Elements

```
<p id="demo" style="display:none">Hello JavaScript!</p>
```

```
<button type="button" onclick="toggleVisibility('demo')">Toggle  
Paragraph</button>
```

```
<script>  
function toggleVisibility(elementId) {  
    var element = document.getElementById(elementId);  
    if (element.style.display === 'none') {  
        element.style.display = 'block';  
    } else {  
        element.style.display = 'none';  
    }  
}  
</script>
```

JavaScript in <head> or <body>

You can place any number of scripts in an HTML document. Scripts can be placed in the `<body>`, or in the `<head>` section of an HTML page, or in both.

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>
</head>
<body>

<h2>Demo JavaScript in Head</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

</body>
</html>
```

Demo JavaScript in Head

A Paragraph.

Try it

Demo JavaScript in Head

Paragraph changed.

Try it

JavaScript in <body>

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo JavaScript in Body</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

<script>
function myFunction() {
  document.getElementById("demo").innerHTML = "Paragraph changed.";
}
</script>

</body>
</html>
```

Demo JavaScript in Body

A Paragraph.

Try it

Demo JavaScript in Body

Paragraph changed.

Try it

External JavaScript

External File: myScript.js

```
function myFunction() {  
    document.getElementById("demo").innerHTML = "Paragraph changed.";  
}
```

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<h2>Demo External JavaScript</h2>
```

```
<p id="demo">A Paragraph.</p>
```

```
<button type="button" onclick="myFunction()">Try it</button>
```

```
<p>This example links to "myScript.js".</p>
```

```
<p>(myFunction is stored in "myScript.js")</p>
```

```
<script src="myScript.js"></script>
```

```
</body>  
</html>
```

Demo External JavaScript

A Paragraph.

Try it

This example links to "myScript.js".

(myFunction is stored in "myScript.js")

Demo External JavaScript

Paragraph changed.

Try it

This example links to "myScript.js".

(myFunction is stored in "myScript.js")

One Click to replace & get original Text

```
<!DOCTYPE html>
<html>
<body>

<h2>Demo External JavaScript</h2>

<p id="demo">A Paragraph.</p>

<button type="button" onclick="myFunction()">Try it</button>

<script src="myScript.js"></script>

</body>
</html>
```

```
var demo =
document.getElementById("demo");
    var originalText = demo.innerHTML;
    var newText = "Paragraph changed";

function myFunction() {
    if (demo.innerHTML ===
originalText) {
        demo.innerHTML = newText;
    } else {
        demo.innerHTML = originalText;
    }
}
```

JavaScript Output

JavaScript can "display" data in different ways:

- Writing into an HTML element, using `innerHTML`.
- Writing into the HTML output using `document.write()`.
- Writing into an alert box, using `window.alert()`.
- Writing into the browser console, using `console.log()`.

Using innerHTML

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My First Paragraph.</p>

<p id="demo"></p>

<script>
document.getElementById("demo").innerHTML = 5 + 6;
</script>

</body>
</html>
```

My First Web Page

My First Paragraph.

11

Using document.write()

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>My First Web Page</h2>  
<p>My first paragraph.</p>
```

```
<p>Never call document.write after the document has finished loading.  
It will overwrite the whole document.</p>
```

```
<script>  
document.write(5 + 6);  
</script>
```

```
</body>  
</html>
```

My First Web Page

My first paragraph.

Never call document.write after the document has finished loading. It will overwrite the whole document.

11|

Using document.write()

Using document.write() after an HTML document is loaded, will delete all existing HTML:

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>My First Web Page</h2>  
<p>My first paragraph.</p>
```

```
<button type="button" onclick="document.write(5 + 6)">Try it</button>
```

```
</body>  
</html>
```

My First Web Page

My first paragraph.

Try it

11

Using window.alert()

```
<!DOCTYPE html>
<html>
<body>

<h2>My First Web Page</h2>
<p>My first paragraph.</p>

<script>
window.alert(5 + 6);
</script>

</body>
</html>
```

We can skip the **window** keyword

An embedded page on this page says

11

OK

My First Web Page

My first paragraph.

Using console.log()

```
<!DOCTYPE html>
<html>
<body>

<h2>Activate Debugging</h2>

<p>F12 on your keyboard will activate debugging.</p>
<p>Then select "Console" in the debugger menu.</p>
<p>Then click Run again.</p>

<script>
console.log(5 + 6);
</script>

</body>
</html>
```

Activate Debugging

F12 on your keyboard will activate debugging.

Then select "Console" in the debugger menu.

Then click Run again.

JavaScript Print

```
<!DOCTYPE html>
<html>
<body>

<h2>The window.print() Method</h2>

<p>Click the button to print the current page.</p>

<button onclick="window.print()">Print this page</button>

</body>
</html>
```

9/9/21, 1:58 PM

TryIt Editor v3.7

The window.print() Method

Click the button to print the current page.

Print this page

The window.print() Method

Click the button to print the current page.

Print this page

Print

1 page

Destination

Save as PDF

Pages

All

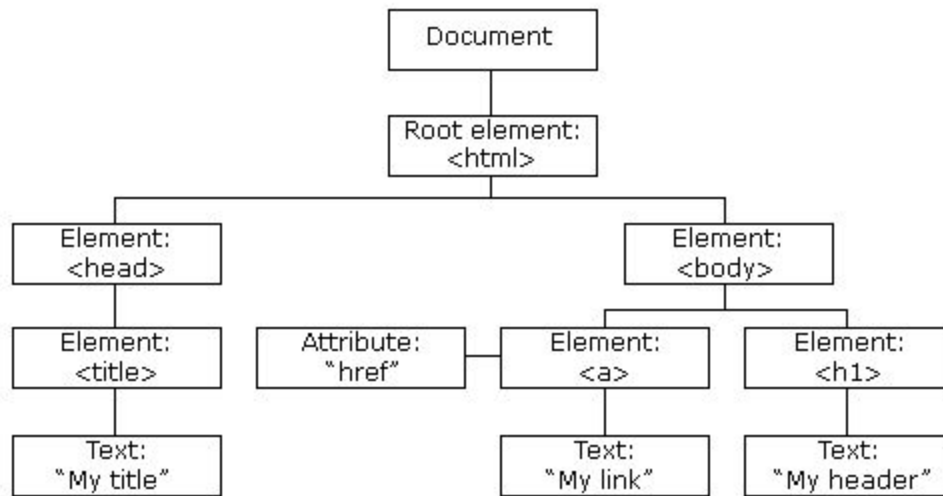
Layout

Landscape

More settings



JavaScript HTML DOM



JavaScript HTML DOM

The HTML DOM model is constructed as a tree of Objects:

With the object model, JavaScript gets all the power it needs to create dynamic HTML:

- JavaScript can change all the HTML elements in the page
- JavaScript can change all the HTML attributes in the page
- JavaScript can change all the CSS styles in the page
- JavaScript can remove existing HTML elements and attributes
- JavaScript can add new HTML elements and attributes
- JavaScript can react to all existing HTML events in the page
- JavaScript can create new HTML events in the page

What is the HTML DOM?

The HTML DOM is a standard object model and programming interface for HTML. It defines:

- The HTML elements as objects
- The properties of all HTML elements
- The methods to access all HTML elements
- The events for all HTML elements

JavaScript Functions

A JavaScript function is a block of code designed to perform a particular task.

A JavaScript function is executed when "something" invokes it (calls it).

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript Functions</h2>
```

```
<p>This example calls a function which performs a calculation, and returns the result:</p>
```

```
<p id="demo"></p>
```

```
<script>
function myFunction(p1, p2) {
  return p1 * p2;
}
document.getElementById("demo").innerHTML = myFunction(4, 3);
</script>
```

```
</body>
</html>
```

JavaScript Functions

This example calls a function which performs a calculation, and returns the result:

12

Function Return

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>JavaScript Functions</h2>
```

```
<p>This example calls a function which performs a calculation and returns the result:</p>
```

```
<p id="demo"></p>
```

```
<script>  
var x = myFunction(4, 3);  
document.getElementById("demo").innerHTML = x;
```

```
function myFunction(a, b) {  
  return a * b;  
}  
</script>
```

```
</body>  
</html>
```

JavaScript Functions

This example calls a function which performs a calculation and returns the result:

12

Why Functions?

You can reuse code: Define the code once, and use it many times.

You can use the same code many times with different arguments, to produce different results.

JavaScript Variables

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Variables</h2>

<p id="demo"></p>

<script>
// Create and display a variable:
let car = "Maruti";
document.getElementById("demo").innerHTML = car;
</script>

</body>
</html>
```

JavaScript Variables

Maruti

JavaScript Objects

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h2>JavaScript Objects</h2>
```

```
<p id="demo"></p>
```

```
<script>  
// Create an object:  
const car = {type:"Maruti", model: 500, color:"white"};  
  
// Display some data from the object:  
document.getElementById("demo").innerHTML = "The car type is " + car.type;  
</script>
```

```
</body>  
</html>
```

JavaScript Objects

The car type is Maruti

The this Keyword

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Objects</h2>
<p>An object method is a function definition, stored as a property value.</p>

<p id="demo"></p>

<script>
// Create an object:
const person = {
  firstName: "Ramesh",
  lastName: "Kumar",
  id: 5566,
  fullName: function() {
    return this.firstName + " " + this.lastName;
  }
};

// Display data from the object:
document.getElementById("demo").innerHTML = person.fullName();
</script>

</body>
</html>
```

JavaScript Objects

An object method is a function definition, stored as a property value.

Ramesh Kumar

JavaScript Events

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>
```

```
<p id="demo"></p>
```

```
</body>  
</html>
```

The time is?

The time is?

Wed Sep 08 2021 15:54:47 GMT+0530 (India Standard Time)

JavaScript Events

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<button onclick="this.innerHTML=Date()">The time is?</button>

</body>
</html>
```

JavaScript HTML Events

The time is?

JavaScript HTML Events

Wed Sep 08 2021 15:58:43 GMT+0530 (India Standard Time)

JavaScript Events

JavaScript code is often several lines long. It is more common to see event attributes calling **functions**:

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript HTML Events</h2>
<p>Click the button to display the date.</p>

<button onclick="displayDate()">The time is?</button>

<script>
function displayDate() {
  document.getElementById("demo").innerHTML = Date();
}
</script>

<p id="demo"></p>

</body>
</html>
```

JavaScript HTML Events

Click the button to display the date.

The time is?

JavaScript HTML Events

Click the button to display the date.

The time is?

Wed Sep 08 2021 16:01:31 GMT+0530 (India Standard Time)

HTML Events

In the following example, an onclick attribute (with code), is added to a <button> element:

```
<button onclick="document.getElementById('demo').innerHTML = Date()">The time is?</button>
```

Here is a list of some common HTML events:

Event	Description
onchange	An HTML element has been changed
onclick	The user clicks an HTML element
onmouseover	The user moves the mouse over an HTML element
onmouseout	The user moves the mouse away from an HTML element
onkeydown	The user pushes a keyboard key
onload	The browser has finished loading the page

OnClick Event

```
<!DOCTYPE html>
<html>
<body>

<button onclick="document.getElementById('demo').innerHTML=Date()">The time is?</button>

<p id="demo"></p>

</body>
</html>
```

The time is?

Mon Apr 17 2023 10:17:52 GMT+0530 (India Standard Time)

onchange Event

```
<!DOCTYPE html>
<html>
<body>
```

```
<p>Select a new car from the list.</p>
```

```
<select id="mySelect" onchange="myFunction()">
  <option value="Audi">Audi</option>
  <option value="BMW">BMW</option>
  <option value="Mercedes">Mercedes</option>
  <option value="Volvo">Volvo</option>
</select>
```

```
<p>When you select a new car, a function is triggered which outputs the value of the selected car.</p>
```

```
<p id="demo"></p>
```

```
<script>
function myFunction() {
  var x = document.getElementById("mySelect").value;
  document.getElementById("demo").innerHTML = "You selected: " + x;
}
</script>
```

```
</body>
</html>
```

Select a new car from the list.

When you select a new car, a function is triggered which outputs the value of the selected car.

Select a new car from the list.



Select a new car, a function is triggered which outputs the value of the selected car.

onmouseover & onmouseout Event

```
<!DOCTYPE html>  
<html>  
<body>
```

```
<h1 onmouseover="style.color='red'"  
onmouseout="style.color='black'">Mouse over this text</h1>
```

```
</body>  
</html>
```

Mouse over this text

Mouse over this text



Onkeydown & onkeyup Event

```
<!DOCTYPE html>
<html>
<head>
<script>
function MyColor(color) {
  document.forms[0].myInput.style.background = color;
}
</script>
</head>
<body>

<form>
Write a message:<br>
<input
type="text"
onkeydown="MyColor('yellow')"
onkeyup="MyColor('white')"
name="myInput">
</form>

</body>
</html>
```

Write a message:

H

Write a message:

Hello

onload Event

```
<!DOCTYPE html>
<html>
<head>
<script>
function myFunction() {
  alert("Page is loaded");
}
</script>
</head>
```

Page is loaded

OK

Hello World!

```
<body onload="myFunction()">
<h2>Hello World!</h2>
</body>

</html>
```

Conditional Statements

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript if .. else</h2>

<p>A time-based greeting:</p>

<p id="demo"></p>

<script>
const time = new Date().getHours();
let greeting;
if (time < 10) {
  greeting = "Good morning";
} else if (time < 20) {
  greeting = "Good day";
} else {
  greeting = "Good evening";
}
document.getElementById("demo").innerHTML = greeting;
</script>

</body>
</html>
```

JavaScript if .. else

A time-based greeting:

Good day

Switch Statement

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript switch</h2>

<p id="demo"></p>

<script>
let day;
switch (new Date().getDay()) {
  case 0:
    day = "Sunday";
    break;
  case 1:
    day = "Monday";
    break;
  case 2:
    day = "Tuesday";
    break;
  case 3:
    day = "Wednesday";
    break;
  case 4:
    day = "Thursday";
    break;
  case 5:
    day = "Friday";
    break;
  case 6:
    day = "Saturday";
    break;
  default:
    day= "Wrong Choice";
}
document.getElementById("demo").innerHTML = "Today is " + day;
</script>

</body>
</html>
```

JavaScript switch

Today is Monday

Different Kinds of Loops

JavaScript supports different kinds of loops:

- **for** - loops through a block of code a number of times
- **for/in** - loops through the properties of an object
- **for/of** - loops through the values of an iterable object
- **while** - loops through a block of code while a specified condition is true
- **do/while** - also loops through a block of code while a specified condition is true

JavaScript Loops-The For Loop

Loops can execute a block of code a number of times.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For Loop</h2>

<p id="demo"></p>

<script>
let text = "";

for (let i = 0; i < 5; i++) {
  text += "The number is " + i + "<br>";
}

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript For Loop

The number is 0
The number is 1
The number is 2
The number is 3
The number is 4

The For/In Loop

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript For In Loop</h2>
<p>The for in statement loops through the properties of an object:</p>

<p id="demo"></p>

<script>
const person = {fname:"John", lname:"Doe", age:25};

let txt = "";
for (let x in person) {
  txt += person[x] + " ";
}

document.getElementById("demo").innerHTML = txt;
</script>

</body>
</html>
```

JavaScript For In Loop

The for in statement loops through the properties of an object:

John Doe 25

The For/Of Loop

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript For Of Loop</h2>
```

```
<p>The for of statement loops through the
values of an iterable object.</p>
```

```
<p id="demo"></p>
```

```
<script>
let language = "JavaScript";
```

```
let text = "";
for (let x of language) {
  text += x + "<br>";
}
```

```
document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

JavaScript For Of Loop

The for of statement loops through the values of an iterable object.

J
a
v
a
S
c
r
i
p
t

```
<!DOCTYPE html>
<html>
<body>
```

```
<h2>JavaScript For Of Loop</h2>
```

```
<p>The for of statement loops through the values of any
iterable object:</p>
```

```
<p id="demo"></p>
```

```
<script>
const cars = ["BMW", "Volvo", "Mini"];
```

```
let text = "";
for (let x of cars) {
  text += x + "<br>";
}
```

```
document.getElementById("demo").innerHTML = text;
</script>
```

```
</body>
</html>
```

JavaScript For Of Loop

The for of statement loops through the values of any iterable object:

BMW
Volvo
Mini

The While Loop

The **while** loop loops through a block of code as long as a specified condition is true.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript While Loop</h2>

<p id="demo"></p>

<script>
let text = "";
let i = 0;
while (i < 10) {
  text += "<br>The number is " + i;
  i++;
}
document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript While Loop

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
```

The Do-While Loop

The **do while** loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true.

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript Do While Loop</h2>

<p id="demo"></p>

<script>
let text = ""
let i = 0;

do {
  text += "<br>The number is " + i;
  i++;
}
while (i < 10);

document.getElementById("demo").innerHTML = text;
</script>

</body>
</html>
```

JavaScript Do While Loop

```
The number is 0
The number is 1
The number is 2
The number is 3
The number is 4
The number is 5
The number is 6
The number is 7
The number is 8
The number is 9
```

Function Sequence

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>Function Sequence</h2>
<p>JavaScript functions are executed in the sequence they are called.</p>

<p id="demo"></p>

<script>
function myDisplay(some) {
    document.getElementById("demo").innerHTML = some;
}

function myFirst() {
    myDisplay("Hello");
}

function mySecond() {
    myDisplay("Goodbye");
}

myFirst();
mySecond();
</script>

</body>
</html>
```

JavaScript Functions

Function Sequence

JavaScript functions are executed in the sequence they are called.

Goodbye

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>Function Sequence</h2>
<p>JavaScript functions are executed in the sequence they are called.</p>

<p id="demo"></p>

<script>
function myDisplay(some) {
    document.getElementById("demo").innerHTML = some;
}

function myFirst() {
    myDisplay("Hello");
}

function mySecond() {
    myDisplay("Goodbye");
}

mySecond();
myFirst();
</script>

</body>
</html>
```

JavaScript Functions

Function Sequence

JavaScript functions are executed in the sequence they are called.

Hello

Sequence Control

call a calculator function (`myCalculator`), save the result, and then call another function (`myDisplayer`) to display the result:

```
<p id="demo"></p>
```

```
<script>
function myDisplayer(some) {
  document.getElementById("demo").innerHTML = some;
}
```

```
function myCalculator(num1, num2) {
  let sum = num1 + num2;
  return sum;
}
```

```
let result = myCalculator(5, 5);
myDisplayer(result);
</script>
```

10

call a calculator function (`myCalculator`), and let the calculator function call the display function (`myDisplayer`):

```
<p id="demo"></p>
```

```
<script>
function myDisplayer(some) {
  document.getElementById("demo").innerHTML =
some;
}
```

```
function myCalculator(num1, num2) {
  let sum = num1 + num2;
  myDisplayer(sum);
}
```

```
myCalculator(5, 5);
</script>
```

What is Callback

- In JavaScript, a callback is a function that is passed as an argument to another function and is intended to be executed after the completion of an asynchronous operation or at a specified time.
- Callbacks are commonly used in scenarios where you want to perform some action once a certain operation, such as a network request, file reading, or a timer, is complete.

```
// Synchronous callback
function doSomething(callback) {
  console.log("Doing something...");
  callback(); // Invoke the callback function
}
```

```
function onComplete() {
  console.log("Operation complete!");
}
```

```
doSomething(onComplete);
```

In this example, the `onComplete` function is passed as a callback to the `doSomething` function. The `doSomething` function performs some operation (in this case, just logging a message) and then invokes the callback function (`onComplete` in this case).

JavaScript Callbacks- A callback is a function passed as an argument to another function

A callback is a function passed as an argument to another function.

```
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>Callback Functions</h2>

<p>The result of the calculation is:</p>
<p id="demo"></p>

<script>
function myDisplayer(something) {
  document.getElementById("demo").innerHTML = something;
}

function myCalculator(num1, num2, myCallback) {
  let sum = num1 + num2;
  myCallback(sum);
}

myCalculator(5, 5, myDisplayer);
</script>

</body>
</html>
```

JavaScript Functions

Callback Functions

The result of the calculation is:

10

myDisplayer is a called a callback function.

It is passed to **myCalculator()** as an argument.

Note

When you pass a function as an argument, **remember not to use parenthesis.**

Right: myCalculator(5, 5, myDisplayer);

Wrong: myCalculator(5, 5, myDisplayer());

Callback-Waiting for a Timeout

```
<!DOCTYPE html>
<html>
<body>

<h1>JavaScript Functions</h1>
<h2>setTimeout() with a Callback</h2>

<p>Wait 3 seconds (3000 milliseconds) for this page to change.</p>

<h1 id="demo"></h1>

<script>
setTimeout(myFunction, 3000);

function myFunction() {
  document.getElementById("demo").innerHTML = "Welcome !!";
}
</script>

</body>
</html>
```

OR

```
<h1 id="demo"></h1>

<script>
setTimeout(function() { myFunction("Welcome !!!"); },
3000);

function myFunction(value) {
  document.getElementById("demo").innerHTML = value;
}
</script>
```

JavaScript Functions

setTimeout() with a Callback

Wait 3 seconds (3000 milliseconds) for this page to change.

Welcome !!

Callback-Waiting for Intervals

<p>Using setInterval() to display the time every second (1000 milliseconds).</p>

<h1 id="demo"></h1>

<script>

setInterval(myFunction, 1000);

```
function myFunction() {  
  let d = new Date();  
  document.getElementById("demo").innerHTML=  
    d.getHours() + ":" +  
    d.getMinutes() + ":" +  
    d.getSeconds();  
}  
</script>
```

10:29:36

JavaScript Promises

"Producing code" is code that can take some time

"Consuming code" is code that must wait for the result

A Promise is a JavaScript object that links producing code and consuming code

The Promise object supports **two properties: state and result.**

JavaScript Promise Object

A JavaScript Promise object contains both the producing code and calls to the consuming code:

Promise Syntax

```
let myPromise = new Promise(function(myResolve, myReject) {  
  // "Producing Code" (May take some time)  
  
  myResolve(); // when successful  
  myReject();  // when error  
});  
  
// "Consuming Code" (Must wait for a fulfilled Promise)  
myPromise.then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

Promise.then() takes two arguments, a callback for success and another for failure.

Both are optional, so you can add a callback for success or failure only.

JavaScript Promise

```
<h2>JavaScript Promise</h2>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function myDisplayer(some) {  
  document.getElementById("demo").innerHTML = some;  
}
```

```
let myPromise = new Promise(function(myResolve, myReject) {  
  let x = 0;
```

```
// some code (try to change x to 5)
```

```
  if (x == 0) {  
    myResolve("OK");  
  } else {  
    myReject("Error");  
  }  
});
```

```
myPromise.then(  
  function(value) {myDisplayer(value);},  
  function(error) {myDisplayer(error);}  
);  
</script>
```

JavaScript Promise

OK

Callback vs Promise Examples

Example Using Callback

<h1>JavaScript Functions</h1>

<h2>setInterval() with a Callback</h2>

<p>Wait 3 seconds (3000 milliseconds) for this page to change.</p>

<h1 id="demo"></h1>

<script>

```
setTimeout(function() { myFunction("WelCome !!!"); }, 3000);
```

```
function myFunction(value) {  
  document.getElementById("demo").innerHTML =  
  value;  
}  
</script>
```

JavaScript Functions

setInterval() with a Callback

Wait 3 seconds (3000 milliseconds) for this page to change.

WelCome !!!

Example Using Promise

<h2>JavaScript Promise</h2>

<p>Wait 3 seconds (3000 milliseconds) for this page to change.</p>

<h1 id="demo"></h1>

<script>

```
const myPromise = new Promise(function(myResolve,  
myReject) {  
  setTimeout(function(){ myResolve("WelCome !!!");  
}, 3000);  
});
```

```
myPromise.then(function(value) {  
  document.getElementById("demo").innerHTML =  
  value;  
});  
</script>
```

JavaScript Promise

Wait 3 seconds (3000 milliseconds) for this page to change.

WelCome !!!

JavaScript Async

"async and await make promises easier to write"

async makes a function return a Promise

await makes a function wait for a Promise

Async Syntax

The keyword **async** before a function makes the function return a promise:

Example

```
async function myFunction() {  
  return "Hello";  
}
```

Is the same as:

```
function myFunction() {  
  return Promise.resolve("Hello");  
}
```

Here is how to use the Promise:

```
myFunction().then(  
  function(value) { /* code if successful */ },  
  function(error) { /* code if some error */ }  
);
```

JavaScript async

Hello

Await Syntax-`async/await`

The `await` keyword can only be used inside an `async` function.

The `await` keyword makes the function pause the execution and wait for a resolved promise before it continues:

```
<h2>JavaScript async / await</h2>
```

```
<h1 id="demo"></h1>
```

```
<script>
```

```
async function myDisplay() {
```

```
  let myPromise = new Promise(function(resolve, reject) {  
    resolve("Welcome !!");
```

```
  });
```

```
  document.getElementById("demo").innerHTML = await myPromise;  
}
```

```
myDisplay();
```

```
</script>
```

JavaScript async / await

Welcome !!

The two arguments (resolve and reject) are pre-defined by JavaScript.

We will not create them, but call one of them when the executor function is ready.

Very often we will not need a reject function.

async/await

Waiting for a Timeout

```
<!DOCTYPE html>
<html>
<body>

<h2>JavaScript async / await</h2>

<p>Wait 3 seconds (3000 milliseconds) for this page to change.</p>

<h1 id="demo"></h1>

<script>
async function myDisplay() {
  let myPromise = new Promise(function(resolve) {
    setTimeout(function() {resolve("Welcome !!");}, 3000);
  });
  document.getElementById("demo").innerHTML = await myPromise;
}

myDisplay();
</script>

</body>
</html>
```

validate

```
var regEmail=/^\w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.\w{2,3})+$/g;
```

```
//Javascript reGex for Email Validation.
```

```
var regPhone=/^\d{10}$/;
```

```
// Javascript reGex for Phone Number validation.
```

```
var regName = /\d+$/g;
```

```
// Javascript reGex for Name validation
```

The background of the slide is a solid blue color. Overlaid on this background are several light blue, semi-transparent rectangular shapes of various sizes and orientations, creating a layered, geometric effect. The text "Thank you" is centered in the middle of the slide in a bold, dark blue, sans-serif font.

Thank you